

LAPORAN PRATIKUM
ALGORITMA DAN PEMROGRAMAN
ABSTRACT DATA TYPE (ADT)

Disusun Oleh:

Khairun Nisa

2511532015

Dosen Pengampu:

DR. WAHYUDI, S.T, M.T

Asisten Pratikum:

Sherly Sukmadira



FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur penulis panjatkan kepada Tuhan Yang Maha Esa atas terselesaikannya laporan praktikum Algoritma dan Pemrograman ini. Laporan ini disusun sebagai bentuk dokumentasi dan pertanggungjawaban atas pelaksanaan praktikum yang telah dilaksanakan pada tanggal 29 Maret 2026. Melalui laporan ini, penulis berharap dapat memberikan gambaran yang jelas mengenai pemahaman konsep ADT serta implementasinya dalam bentuk kode program.

Penulis mengucapkan terima kasih kepada Bapak Dosen serta asisten laboratorium yang telah membimbing selama praktikum berlangsung. Penulis menyadari sepenuhnya bahwa laporan ini masih jauh dari kata sempurna, baik dari segi penyusunan, bahasa, maupun kedalaman materi. Oleh karena itu, penulis dengan rendah hati menerima segala kritik dan saran yang membangun demi perbaikan dan penyempurnaan laporan di masa mendatang. Semoga laporan ini dapat bermanfaat bagi pembaca dan menjadi referensi untuk perkembangan ilmu pengetahuan di teknologi informasi.

Padang, 31 Maret 2026

Penulis

DAFTAR ISI

KATA PENGANTAR	i
DAFTAR ISI	ii
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan Pratikum	2
1.3 Manfaat Pratikum	2
BAB II PEMBAHASAN	3
2.1 Pembuatan Package dan Class Baru.....	3
2.2 Pembahasan Kelas Jam_2511532015	3
2.3 Pengujian Statis dengan JamDriver_2511532015	5
2.4 Pengujian Interaktif dengan JamDriver_2511532015.....	6
BAB III KESIMPULAN	9
3.1 Kesimpulan	9
DAFTAR PUSTAKA	10

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam pengembangan perangkat lunak modern, efesiesi pengelolaan data merupakan faktor kritis yang menentukan kinerja suatu aplikasi, yang menuntut tidak hanya memahami sintaks pemrograman, tetapi juga mampu merancang Solusi perangkat lunak ang efisien, skalabel, dan mudah dipelihara. Salah satu fondasi utama dalam mencapai komptensi tersebut adalah penguasaan terhadap struktur data dan konsep Abstract Data Type (ADT). ADT berfungsi sebagai kerangka konseptual untuk mendefenisikan struktur data berdasarkan operasinya. ADT memberikan tingkat abstraksi yang lebih tinggi bagi pengembang, memungkinkan mereka untuk fokus pada makna mendasar dari data dan kemampuan manipulasinya, daripada kerumitan implementasinya [1].

Dalam ADT semua detail internalnya disembunyikan dari pengguna, yang memungkinkan fleksibilitas dan modularitas dalam pemrograman [2]. Pemisah antara interface dan implementasi ini merupakan prinsip dasar dalam rekayasa perangkat lunak. Dalam konteks Pendidikan ilmu komputer, pemahaman yang mendalam mengenai ADT sangat penting agar mahasiswa mampu membedakan antara spesifikasi logis (apa yang dilakukan struktur data) dan implementasi konkret (bagaimana struktur data tersebut dibangun).

Pada konteks pembelajaran, praktikum Struktur Data merupakan sarana strategis untuk menginternalisasi konsep ADT melalui pemahaman langsung. Mahasiswa tidak hanya ditugaskan untuk menulis kode, tetapi juga menganalisis trade-off antara berbagai pendekatan implementasi, dimana proses ini melatih pola pikir komputasional yang esensial dalam pemecahan masalah teknik di dunia nyata.

1.2 Tujuan Praktikum

1. Mahasiswa mampu memahami perbedaan mendasar antara spesifikasi logis (antarmuka) dan representasi fisik (implementasi) dari sebuah tipe data.
2. Mahasiswa mampu mengkodekan operasi-operasi dasar ADT menggunakan bahasa pemrograman yang ditentukan secara benar dan efisien.
3. Mahasiswa mampu melakukan pengujian terhadap program untuk memastikan bahwa implementasi ADT berperilaku sesuai dengan definisi matematisnya tanpa kesalahan.
4. Mahasiswa mampu menyusun kode yang termodularisasi, sehingga bagian implementasi data dapat dipisahkan dari logika utama program.

1.3 Manfaat Praktikum

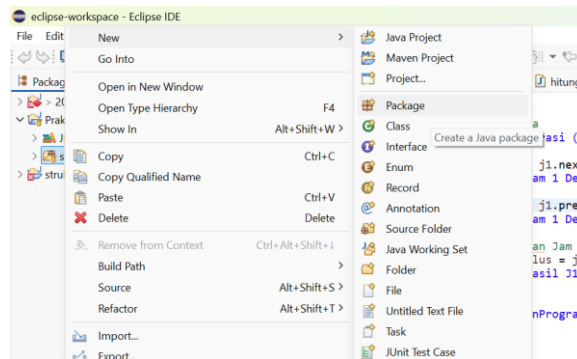
1. Meningkatkan keterampilan pemrograman dalam mengelola memori dan struktur data secara manual maupun menggunakan fitur bahasa pemrograman.
2. Melatih pola pikir yang sistematis dalam memecahkan masalah yang berkaitan dengan organisasi data dan efisiensi algoritma.
3. Memberikan fondasi yang kuat untuk mata kuliah lanjutan.

BAB II

PEMBAHASAN

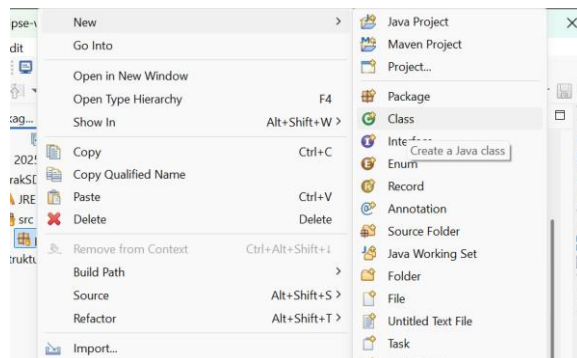
2.1 Pembuatan Package dan Kelas Baru

2.1.1 Buat package baru dengan nama 'pekan1_2511532015'



Gambar 2.1.1: Pembuatan Package baru

2.1.2 Setelah package berhasil dibuat, buat Kelas Java yang akan berada dalam package tersebut. Kelas ini nantinya akan berfungsi sebagai wadah untuk menuliskan seluruh kode implementasi ADT.



Gambar 1.2.2: Pembuatan Kelas di dalam Package pekan1_2511532015

2.2 Pembahasan Kelas Jam_2511532015

Kelas `Jam_2511532015` merupakan implementasi dari ADT yang merepresentasikan data waktu dalam format 24 jam (hh:mm:ss). Kelas ini berfungsi sebagai cetak biru (blueprint) yang mendefinisikan atribut data serta operasi-operasi yang dapat dilakukan terhadap objek jam, dengan menyembunyikan detail implementasi internalnya dari pengguna.

```

1 package pekan_2511532015;
2
3 public class Jam_2511532015 {
4     private int hh; //0..23
5     private int mm; //0..59
6     private int ss; //0..59
7
8     //--- Validator ---
9     public static boolean isValid(int h, int m, int s) {
10         return (0 <= h && h < 24) && (0 <= m && m < 60) && (0 <= s && s < 60);
11     }
12
13     //--- Konstruktor/Kreator ---
14     public Jam_2511532015(int h, int m, int s) {
15         this.hh = h; this.mm = m; this.ss = s;
16     }
17
18     //--- Selektor ---
19     public int getHour() {return hh;}
20     public int getMinute() {return mm;}
21     public int getSecond() {return ss;}
22
23     //--- Mutator ---
24     public void setHour(int h) {this.hh = h;}
25     public void setMinute(int m) {this.mm = m;}
26     public void setSecond(int s) {this.ss = s;}
27
28     //--- Konversi ---
29     public int toSeconds() {return hh * 3600 + mm * 60 + ss;}
30     public static Jam_2511532015 fromSeconds(int total) {
31         if (total < 0) throw new IllegalArgumentException("detik negatif");
32         total %= 24 * 3600; //wrap 24 jam
33         int h = total / 3600; total %= 3600;
34         int m = total / 60; int s = total % 60;
35         return new Jam_2511532015(h, m, s);
36     }
37
38     //--- Relasional ---
39     public int compareTo(Jam_2511532015 other) {return Integer.compare(this.toSeconds(), other.toSeconds());}
40     public boolean equals(Object o) {
41         if (!(o instanceof Jam_2511532015)) return false;
42         return hh == j.hh && mm == j.mm && ss == j.ss;
43     }
44     public int hashCode() {return java.util.Objects.hash(hh, mm, ss);}
45
46     //--- Aritmatika ---
47     public Jam_2511532015 plus(Jam_2511532015 other) {return fromSeconds(this.toSeconds() + other.toSeconds());}
48     public Jam_2511532015 minus(Jam_2511532015 other) {return fromSeconds(Math.floorMod(this.toSeconds() - other.toSeconds(), 24 * 3600));}
49     public Jam_2511532015 nextSecond() {return fromSeconds(this.toSeconds() + 1);}
50     public Jam_2511532015 nextSecond(int n) {return fromSeconds(this.toSeconds() + Math.max(0, n));}
51     public Jam_2511532015 prevSecond() {return fromSeconds(Math.floorMod(this.toSeconds() - 1, 24 * 3600));}
52     public Jam_2511532015 prevSecond(int n) {return fromSeconds(Math.floorMod(this.toSeconds() - Math.max(0, n), 24 * 3600));}
53
54     //--- Durasi (detik) --- bisa negatif jika this > other? sesuai spesifikasi durasi (30w, 30kh)
55     public static int durasiDetik(Jam_2511532015 j1w, Jam_2511532015 j2kh) {return j1w.toSeconds() - j2kh.toSeconds();}
56     //overriding ke string
57     public String toString() {return String.format("%02d:%02d:%02d", hh, mm, ss);}
58
59 }
60

```

Gambar 2.2.1: Kode Program lengkap Clas Jam_2511532015

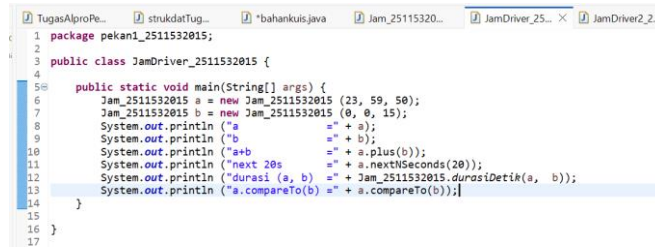
Kelas ini dirancang sebagai modul independent yang menyimpan struktur data dan logika operasional terkait waktu, namun tidak dimaksudkan untuk dieksekusi secara langsung yang berarti kelas ini semata-mata untuk menyediakan fondasi bagi kelas lain yang akan menggunakannya.

Seluruh atribut penyimpanan data, yaitu jam, menit, dan detik, dideklarasikan sebagai private. Untuk menjaga integritas data, kelas ini menyediakan metode validator statis yang memastikan nilai jam berada pada rentang 0-23, menit 0-59, dan detik 0-59.

Metode-metode yang terdapat di dalam kelas ini, seperti konstruktor, setter, dan getter, disiapkan untuk diinovasikan oleh kelas lain. Metode *setter* berfungsi sebagai pintu masuk untuk memberikan nilai awal atau mengubah nilai objek, sementara metode *getter* berfungsi sebagai pintu keluar untuk mengambil nilai tersebut saat dibutuhkan. Selain itu, metode-metode logika seperti validasi, konversi, operasi aritmatika, dan perbandingan juga didefinisikan di sini sebagai layanan siap pakai. Kelas ini tidak melakukan proses input maupun output secara langsung; seluruh fungsionalitasnya baru akan aktif ketika objek dari kelas ini dibuat (*instansiasi*) dan metode-metodenya dipanggil oleh kelas penggerak (*driver class*), seperti `JamDriver_2511532015` atau `JamDriver2_2511532015`.

2.3 Pengujian Statis dengan JamDriver_2511532015

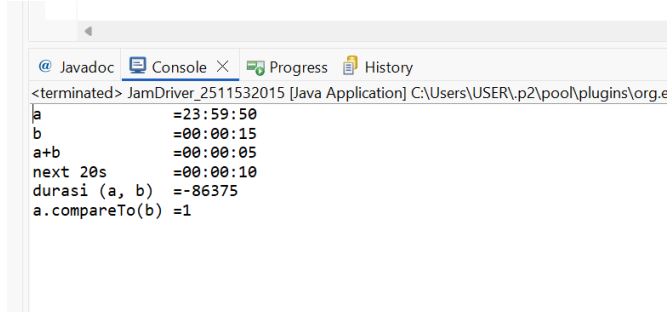
Kelas JamDriver_2511532015 berfungsi sebagai saran pengujian awal (static testing) untuk memverifikasi kebenaran logika yang telah dibangun pada kelas Jam_2511532015.



```
1 package pekan1_2511532015;
2
3 public class JamDriver_2511532015 {
4
5     public static void main(String[] args) {
6         Jam_2511532015 a = new Jam_2511532015 (23, 59, 50);
7         Jam_2511532015 b = new Jam_2511532015 (0, 0, 15);
8         System.out.println ("a      =" + a);
9         System.out.println ("b      =" + b);
10        System.out.println ("a+b     =" + a.plus(b));
11        System.out.println ("next 20s =" + a.nextNSeconds(20));
12        System.out.println ("durasi (a, b) =" + Jam_2511532015.durasiDetik(a, b));
13        System.out.println ("a.compareTo(b) =" + a.compareTo(b));
14    }
15 }
16
17 }
```

Gambar 2.3.1: Kode Program Pengujian Statis dengan JamDriver_2511532015

Pengujian ini dilakukan tanpa interaksi pengguna, di mana nilai input objek jam ditentukan secara langsung di dalam kode program. Pada eksekusi program, dilakukan berbagai operasi dasar seperti penjumlahan dua objek jam, penambahan detik tertentu, perhitungan durasi, dan perbandingan waktu.



```
<terminated> JamDriver_2511532015 [Java Application] C:\Users\USER\p2\pool\plugins\org.e
a      =23:59:50
b      =00:00:15
a+b     =00:00:05
next 20s =00:00:10
durasi (a, b) =-86375
a.compareTo(b) =1
```

Gambar 2.3.2: Hasil output dari kelas JamDriver_2511532015

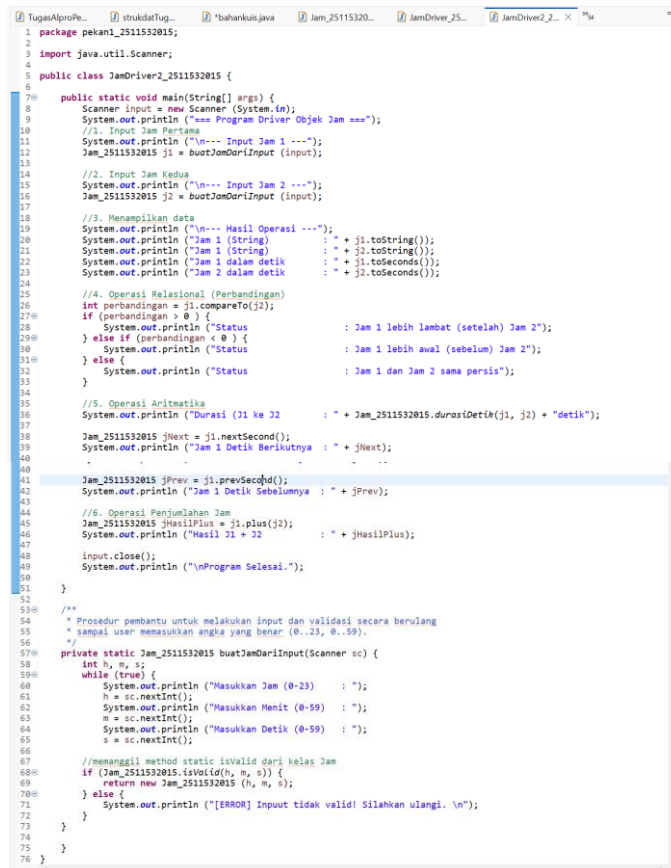
Berdasarkan hasil eksekusi program, terlihat bahwa objek a berhasil diinisialisasikan dengan nilai waktu 23:59:50, sedangkan objek b diinisialisasikan dengan nilai 00:00:15. Pada operasi penjumlahan $a + b$, program menghasilkan keluaran 00:00:05. Hasil ini menunjukkan bahwa mekanisme wrap-around 24 jam telah berfungsi dengan baik. Penjumlahan 23:59:50 dengan 00:00:15 secara logika akan melewati batas tengah malam, sehingga hasil perhitungan dimodulo dengan total detik dalam satu hari (86400 detik) dan menghasilkan waktu 00:00:05 di hari berikutnya.

Selanjutnya, operasi `nextSeconds(20)` yang dipanggil pada objek `a` menghasilkan nilai `00:00:10` yang membuktikan navigasi waktu maju sebanyak 20 detik dari `23:59:50`.

Untuk operasi durasi, hasil yang diperoleh adalah `-86375` detik, nilai negative muncul karena metode `durasiDetik` menghitung selisih linear `b-a` dalam konteks waktu yang sama. Dan yang terakhir metode `CompareTo` menghasilkan nilai `1` (positif), yang menunjukkan bahwa objek `a` dianggap lebih besar atau lebih akhir dibandingkan dengan objek `b` dalam perbandingan waktu pada hari yang sama.

2.4 Pengujian Interaktif dengan `JamDriver_2511532015`

Kelas ini merupakan pengembangan dari kelas pengujian statis yang menambahkan fitur interaksi langsung dengan pengguna. Tujuan utama kelas ini adalah untuk memvalidasi ketahanan program ketika menghadapi variasi input dari pengguna serta menguji mekanisme validasi data yang telah dibangun pada kelas `Jam_2511532015`.

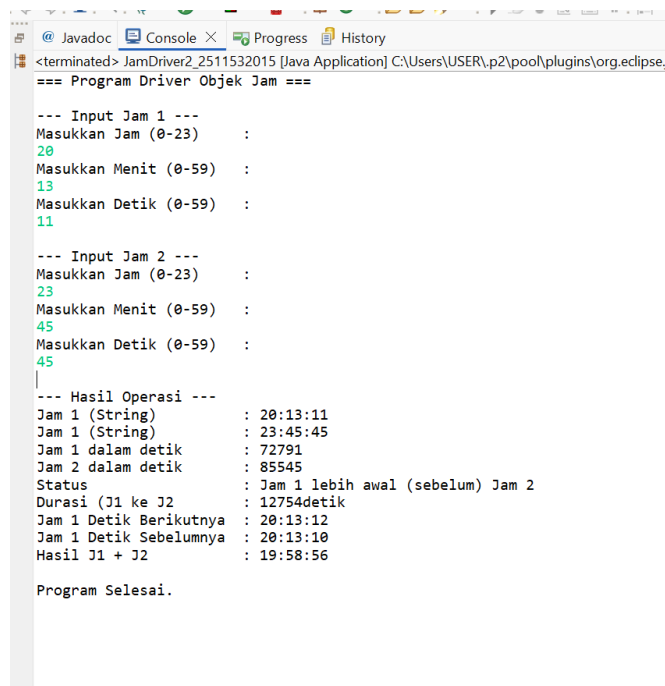


```
1 package pekan1_2511532015;
2
3 import java.util.Scanner;
4
5 public class JamDriver_2511532015 {
6
7     public static void main(String[] args) {
8         Scanner input = new Scanner(System.in);
9         System.out.println("=== Program Driver Objek Jam ===");
10        //1. Input Jam Pertama
11        System.out.println("\n--- Input Jam 1 ---");
12        Jam_2511532015 j1 = buatJamDariInput(input);
13
14        //2. Input Jam Kedua
15        System.out.println("\n--- Input Jam 2 ---");
16        Jam_2511532015 j2 = buatJamDariInput(input);
17
18        //3. Menampilkan data
19        System.out.println("\n--- Hasil Operasi ---");
20        System.out.println("Jam 1 (String) : " + j1.toString());
21        System.out.println("Jam 1 (String) : " + j2.toString());
22        System.out.println("Jam 1 dalam detik : " + j1.toSeconds());
23        System.out.println("Jam 2 dalam detik : " + j2.toSeconds());
24
25        //4. Operasi Relasional (Perbandingan)
26        int perbandingan = j1.compareTo(j2);
27        if (perbandingan > 0) {
28            System.out.println("Status : Jam 1 lebih lambat (setelah) Jam 2");
29        } else if (perbandingan < 0) {
30            System.out.println("Status : Jam 1 lebih awal (sebelum) Jam 2");
31        } else {
32            System.out.println("Status : Jam 1 dan Jam 2 sama persis");
33        }
34
35        //5. Operasi Aritmatika
36        System.out.println("Durasi (J1 ke J2 : " + Jam_2511532015.durasiDetik(j1, j2) + "detik");
37        Jam_2511532015 jNext = j1.nextSecond();
38        System.out.println("Jam 1 Detik Berikutnya : " + jNext);
39
40        Jam_2511532015 jPrev = j1.prevSecond();
41        System.out.println("Jam 1 Detik Sebelumnya : " + jPrev);
42
43        //6. Operasi Penjumlahan Jam
44        Jam_2511532015 jHasilPlus = j1.plus(j2);
45        System.out.println("Hasil J1 + J2 : " + jHasilPlus);
46        input.close();
47        System.out.println("\nProgram Selesai.");
48    }
49
50    /**
51     * Prosedur pembantu untuk melakukan input dan validasi secara berulang
52     * sampai user memasukkan angka yang benar (0..23, 0..59).
53     */
54    private static Jam_2511532015 buatJamDariInput(Scanner sc) {
55        int h, m, s;
56        while (true) {
57            System.out.println("Masukkan Jam (0-23) : ");
58            h = sc.nextInt();
59            System.out.println("Masukkan Menit (0-59) : ");
60            m = sc.nextInt();
61            System.out.println("Masukkan Detik (0-59) : ");
62            s = sc.nextInt();
63
64            //memanggil method static isValid dari kelas Jam
65            if (Jam_2511532015.isValid(h, m, s)) {
66                return new Jam_2511532015(h, m, s);
67            } else {
68                System.out.println("[ERROR] Input tidak valid! Silahkan ulangi. \n");
69            }
70        }
71    }
72 }
73
74
75
76 }
```

Gambar 2.4.1: Kode Program kelas `JamDriver2_2511532015`

Program dimulai dengan memanggil metode bantu `buatJamDariInput` sebanyak dua kali untuk menghasilkan objek jam pertama dan kedua. Pada metode ini, diterapkan mekanisme perulangan (looping) yang tidak akan berhenti sebelum pengguna memasukkan nilai jam, menit, dan detik yang valid.

Setelah kedua objek jam berhasil dibuat, program melanjutkan ke tahap proses keluaran. Berbagai operasi ADT dipanggil dan hasilnya akan ditampilkan secara berurutan pada konsol. Operasi yang diuji meliputi konversi waktu ke satuan detik, perbandingan relasional untuk menentukan urutan waktu, perhitungan durasi antara dua waktu, navigasi waktu (maju dan mundur satu detik), serta penjumlahan dua objek jam.



```

@ Javadoc Console X Progress History
<terminated> JamDriver2_2511532015 [Java Application] C:\Users\USER\p2\pool\plugins\org.eclipse.j
=== Program Driver Objek Jam ===

--- Input Jam 1 ---
Masukkan Jam (0-23) : 20
Masukkan Menit (0-59) : 13
Masukkan Detik (0-59) : 11

--- Input Jam 2 ---
Masukkan Jam (0-23) : 23
Masukkan Menit (0-59) : 45
Masukkan Detik (0-59) : 45

|
--- Hasil Operasi ---
Jam 1 (String) : 20:13:11
Jam 1 (String) : 23:45:45
Jam 1 dalam detik : 72791
Jam 2 dalam detik : 85545
Status : Jam 1 lebih awal (sebelum) Jam 2
Durasi (J1 ke J2) : 12754detik
Jam 1 Detik Berikutnya : 20:13:12
Jam 1 Detik Sebelumnya : 20:13:10
Hasil J1 + J2 : 19:58:56

Program Selesai.

```

Gambar 2.4.2: Hasil output kelas `JamDriver2_2511532015`

Pengguna memasukkan nilai waktu secara manual melalui keyboard lalu diproses melalui metode `buatJamDariInput` dan langsung diterima oleh program.

Pada hasil operasi, metode `toString()` menampilkan representasi string dari kedua objek jam, mengkonversi ke detik, metode `durasiDetik` menghasilkan selisih detik antara kedua waktu yang diinputkan. Operasi perbandingan `CompareTo` menghasilkan status bahwa Jam 1 lebih awal dari

Jam 2 yang merefleksikan perbandingan nilai total detik dari masing-masing objek.

Operasi navigasi waktu menampilkan 20:13:12 untuk penambahan satu detik dan 20:13:10 untuk pengurangan satu detik pada Jam 1. Sementara itu, operasi penjumlahan $J1 + J2$ menghasilkan 19:58:56, yang merupakan hasil dari penjumlahan total detik kedua objek yang kemudian dikonversi kembali ke format jam dengan penerapan modulo 24 jam. Output tersebut merepresentasikan hasil pemanggilan seluruh metode publik yang tersedia pada kelas `Jam_2511532015`.

BAB III

KESIMPULAN

3.1 Kesimpulan

Praktikum Struktur Data dengan topik Abstract Data Type (ADT) ini membantu memahami konsep pemisahan antara definisi type data dan implementasinya melalui pembuatan kelas `Jam_2511532015`. Kelas ini berperan sebagai cetak biru yang menyimpan atribut waktu serta operasi yang dapat dilakukan, tanpa memiliki metode utama untuk dieksekusi secara langsung.

Pengujian yang dilakukan melalui kelas driver menunjukkan bahwa seluruh metode dapat dipanggil dan menghasilkan output yang sesuai dengan perhitungan manual. Secara keseluruhan, kegiatan praktikum ini menegaskan manfaat penggunaan ADT dalam rekayasa perangkat lunak. Dengan memisahkan logika data dari eksekusi program, kode menjadi lebih terstruktur, modular, dan mudah dikembangkan. Mahasiswa tidak hanya belajar menulis kode yang berjalan, tetapi juga merancang komponen perangkat lunak yang memiliki integritas data tinggi dan siap digunakan dalam sistem yang lebih kompleks.

DAFTAR PUSTAKA

- [1] deepchecks, "Tipe Data Abstrak," deepchecks, [Online]. Available: [https://deepchecks.com/glossary/abstract-data-type/#:~:text=Type%20data%20abstrak%20\(ADT\)%20adalah,sebagai%20landasan%20untuk%20pengkodean%20modular..](https://deepchecks.com/glossary/abstract-data-type/#:~:text=Type%20data%20abstrak%20(ADT)%20adalah,sebagai%20landasan%20untuk%20pengkodean%20modular..) [Accessed 31 Maret 2026].
- [2] A. Verma, "Tipe Data Abstrak dalam Struktur Data," intellipaat, [Online]. Available: <https://intellipaat.com/blog/abstract-data-type/#:~:text=Apa%20itu%20Type%20Data%20Abstrak%20dalam%20Struktur%20Data?,%2C%20queue%2C%20dan%20lain%20sebagainya..> [Accessed 31 Maret 2026].

Laporan Tugas ADT Pekan 1

1. Soal:

Tugas Praktikum Pekan 1

- Buatlah sebuah program ADT mobil yang memiliki atribut: nama, tahun, cc, harga, dan merk
- Contoh: Avanza, 2020, 1300, 1500000000, Toyota
- Buatlah method ADT mobil tersebut: tambah mobil, hapus mobil, Selektor dan mutator
- Buatlah 2 kelas: 1 kelas ADT mobil dan 1 kelas driver. Setiap kelas lengkapi dengan nim

2. Penjelasan mengenai kelas Mobil_2511532015

```
package pekan1_2511532015;

public class Mobil_2511532015 {
    private String nama;
    private int tahun;
    private int cc;
    private int harga;
    private String merek;

    //--- Array Biasa ---
    private static Mobil_2511532015[] garasi = new
Mobil_2511532015[10];
    private static int jumlahMobil = 0;

    //--- Konstruktor ---
    public Mobil_2511532015 (String nama, int tahun, int cc, int
harga, String merek) {
        this.nama = nama;
        this.tahun = tahun;
        this.cc = cc;
        this.harga = harga;
        this.merek = merek;
    }

    //--- Selector ---
    public String getNama() {return nama;}
    public int getTahun() {return tahun;}
    public int getCC() {return cc;}
    public int getHarga() {return harga;}
    public String getMerek() {return merek;}

    //dapatin jumlah mobil
    public static int getJumlahMobil() {
```

```

        return jumlahMobil;
    }
    //dapatin kapasitas garasi
    public static int getKapasitasGarasi() {
        return garasi.length;
    }

    //--- Mutator ---
    public void setNama(String nama) {this.nama = nama;}
    public void setTahun(int tahun) {this.tahun = tahun;}
    public void setCC(int cc) {this.cc = cc;}
    public void setHarga(int harga) {this.harga = harga;}
    public void setMerek(String merek) {this.merek = merek;}

    //--- menampilkan data mobil ---
    public static void tampilkanSemuaMobil() {
        System.out.println("=== Daftar Mobil ===");
        if (jumlahMobil == 0) {
            System.out.println ("Garasi masih kosong");
        } else {
            for (int i= 0; i< jumlahMobil; i++) {
                Mobil_2511532015 m = garasi[i];
                System.out.println ("Mobil ke-"+ (i+1)+" : ");
                System.out.println ("Nama: " + m.nama);
                System.out.println ("Tahun: " + m.tahun);
                System.out.println ("CC: " + m.cc);
                System.out.println ("Harga: "+m.harga);
                System.out.println ("Merek: "+m.merek);
                System.out.println
(" \n=====");
            }
        }
    }

    //--- Tambah Mobil ---
    public static boolean tambahMobil (Mobil_2511532015 mobilBaru) {
        if (mobilBaru == null) {
            System.out.println ("Mobil tidak boleh kosong");
            return false;
        }
        if (jumlahMobil >= garasi.length) {
            System.out.println ("Garasi sudah penuh
(maksimal)");
            return false;
        }
        garasi[jumlahMobil] = mobilBaru;
        jumlahMobil++;
        System.out.println("Mobil berhasil ditambahkan");
        return true;
    }

    //--- Hapus Mobil ---
    public static boolean hapusMobil(String namaMobil) {
        int index = -1;
        for (int i = 0; i < jumlahMobil; i++) {
            if (garasi[i].nama.equalsIgnoreCase(namaMobil)) {
                index = i;
                break;
            }
        }
    }

```

```

    }
    }
    if (index == -1) {
        System.out.println ("Mobil tidak ditemukan");
        return false;
    }
    for (int i = index; i < jumlahMobil - 1; i++) {
        garasi[i]=garasi[i+1];
    }
    garasi [jumlahMobil - 1]= null;
    jumlahMobil--;
    System.out.println ("Berhasil menghapus mobil");
    return true;
}
}

```

Program dirancang untuk mengelola data mobil dalam sebuah garasi menggunakan array statis sebagai struktur penyimpanan utama. Pada kelas utama Mobil_2511532015, berfungsi sebagai cetakan objek yang mendefinisikan atribut-atribut mobil seperti nama, tahun, cc, harga, dan merek. Untuk menyimpan koleksi mobil digunakan array statis (biasa) berukuran tetap. Kelas ini dilengkapi konstruktor untuk inisialisasi objek, metode selector dan mutator untuk akses data yang aman, serta metode operasional utama yaitu untuk menambahkan mobil dan menghapus mobil.

3. Penjelasan mengenai MobilDriver_2511532015

```

4. package pekan1_2511532015;
5.
6. public class Mobil_2511532015 {
7.     private String nama;
8.     private int tahun;
9.     private int cc;
10.    private int harga;
11.    private String merek;
12.
13.    ///--- Array Biasa ---
14.    private static Mobil_2511532015[] garasi = new
Mobil_2511532015[10];
15.    private static int jumlahMobil = 0;
16.
17.    ///--- Konstruktor ---
18.    public Mobil_2511532015 (String nama, int tahun, int cc, int
harga, String merek) {
19.        this.nama = nama;
20.        this.tahun = tahun;
21.        this.cc = cc;

```

```

22.         this.harga = harga;
23.         this.merek = merek;
24.     }
25.
26.     //--- Selector ---
27.     public String getNama() {return nama;}
28.     public int getTahun() {return tahun;}
29.     public int getCC() {return cc;}
30.     public int getHarga() {return harga;}
31.     public String getMerek() {return merek;}
32.
33.     //dapatin jumlah mobil
34.     public static int getJumlahMobil() {
35.         return jumlahMobil;
36.     }
37.     //dapatin kapasitas garasi
38.     public static int getKapasitasGarasi() {
39.         return garasi.length;
40.     }
41.
42.     //--- Mutator ---
43.     public void setNama(String nama) {this.nama = nama;}
44.     public void setTahun(int tahun) {this.tahun = tahun;}
45.     public void setCC(int cc) {this.cc = cc;}
46.     public void setHarga(int harga) {this.harga = harga;}
47.     public void setMerek(String merek) {this.merek = merek;}
48.
49.     //--- menampilkan data mobil ---
50.     public static void tampilkanSemuaMobil() {
51.         System.out.println("=== Daftar Mobil ===");
52.         if (jumlahMobil == 0) {
53.             System.out.println ("Garasi masih kosong");
54.         } else {
55.             for (int i= 0; i< jumlahMobil; i++) {
56.                 Mobil_2511532015 m = garasi[i];
57.                 System.out.println ("Mobil ke-"+ (i+1)+" : ");
58.                 System.out.println ("Nama: " + m.nama);
59.                 System.out.println ("Tahun: " + m.tahun);
60.                 System.out.println ("CC: " + m.cc);
61.                 System.out.println ("Harga: "+m.harga);
62.                 System.out.println ("Merek: "+m.merek);
63.                 System.out.println
64.                 ("\\n=====");
65.             }
66.         }
67.     //--- Tambah Mobil ---
68.     public static boolean tambahMobil (Mobil_2511532015 mobilBaru) {
69.         if (mobilBaru == null) {
70.             System.out.println ("Mobil tidak boleh kosong");
71.             return false;
72.         }
73.         if (jumlahMobil >= garasi.length) {
74.             System.out.println ("Garasi sudah penuh
75.             (maksimal)");
76.             return false;
77.         }

```

```

77.         garasi[jumlahMobil] = mobilBaru;
78.         jumlahMobil++;
79.         System.out.println("Mobil berhasil ditambahkan");
80.         return true;
81.     }
82.
83.     //--- Hapus Mobil ---
84.     public static boolean hapusMobil(String namaMobil) {
85.         int index = -1;
86.         for (int i = 0; i < jumlahMobil; i++) {
87.             if (garasi[i].nama.equalsIgnoreCase(namaMobil)) {
88.                 index = i;
89.                 break;
90.             }
91.         }
92.         if (index == -1) {
93.             System.out.println ("Mobil tidak ditemukan");
94.             return false;
95.         }
96.         for (int i = index; i < jumlahMobil - 1; i++) {
97.             garasi[i]=garasi[i+1];
98.         }
99.         garasi [jumlahMobil - 1]= null;
100.        jumlahMobil--;
101.        System.out.println ("Berhasil menghapus mobil");
102.        return true;
103.
104.    }
105.
106. }

```

Kelas ini berperan sebagai pengendali program sebelumnya yang berisi metode main sebagai titik eksekusi utama, dimana bagian awal program dilakukan pengujian otomatis menggunakan data hardcoded untuk memastikan metode dasar berjalan dengan baik.

Selanjutnya, pada bagian simulasi (dealer mobil), merupakan implementasi interaksi antara pengguna dan sistem yang mengubah program sekedar kode yang berjalan otomatis menjadi simulasi sistem manajemen showroom mobil. Dimana terdapat 4 pilihan yang dapat dipilih oleh user (yang mengatur showwoom mobil) dimana pilihan tersebut antara lain, menampilkan semua stok mobil, menambahkan mobil baru yang dianggap sebagai barang dating ke gudang dimana nantinya akan dimintai inputan dari user mengenai data-data mobil, menghapus mobil yang dianggap sebagai unit terjual kepada pelanggan, dan terakhir pilihan untuk mengakhiri simulasi.

4. Penjelasan mengenai Output kelas MobilDriver_2511532015

```
<terminated> MobilDriver_2511532015 [Java Application] C:\Users\USER\p2\poo\plugins\org.eclipse.justj.open
=== Program ADT Mobil ===
<<< Menambahkan mobil >>>
Mobil berhasil ditambahkan
Mobil berhasil ditambahkan
Mobil berhasil ditambahkan
=== Daftar Mobil ===
Mobil ke-1:
Nama: Civic
Tahun: 2019
CC: 1500
Harga: 250000000
Merek: Honda

=====
Mobil ke-2:
Nama: Pajero
Tahun: 2021
CC: 2400
Harga: 500000000
Merek: Mitsubishi

=====
Mobil ke-3:
Nama: Brio
Tahun: 2022
CC: 1200
Harga: 180000000
Merek: Honda

=====
<<< Hapus Mobil >>>
Hapus mobil ke-2
Berhasil menghapus mobil
=== Daftar Mobil ===
Mobil ke-1:
Nama: Civic
Tahun: 2019
CC: 1500
Harga: 250000000
Merek: Honda
```

Ketika pertama kali program dijalankan, output yang dihasilkan adalah bagian pengujian otomatis dimana sistem akan mencatat pesan bahwa mobil sedang proses menambahkan dan diikuti dengan pesan suksse menambahkan mobil sehingga pengguna dapat melihat bahwa ada 3 mobil yang telah dimasukkan ke dalam array garasi. Selanjutnya sistem akan mendemontrasikan fitur penghapusan menggunakan metode hapus mobil lalu menampilkan kembali daftar mobil yang tersedia setelah dihapus.

```

=== Simulasi Dealer ===
Stok: 2/10 mobil
1. Lihat stok mobil
2. Mobil baru datang (tambah stok)
3. Pelanggan membeli mobil (Hapus stok)
4. Keluar:
Pilih menu:
1
Stok Mobil Tersedia
=== Daftar Mobil ===
Mobil ke-1:
Nama: Civic
Tahun: 2019
CC: 1500
Harga: 250000000
Merek: Honda

=====
Mobil ke-2:
Nama: Brio
Tahun: 2022
CC: 1200
Harga: 180000000
Merek: Honda

=====

```

Jika pengguna menginputkan angka 1, maka sistem akan langsung menampilkan stok mobil yang ada saat ini .

```

=== Simulasi Dealer ===
Stok: 2/10 mobil
1. Lihat stok mobil
2. Mobil baru datang (tambah stok)
3. Pelanggan membeli mobil (Hapus stok)
4. Keluar:
Pilih menu:
2

Stok mobil baru datang
Nama Mobil: Pajero
Tahun: 2022
CC: 5000
Harga: 900000000
Merek: Mamasuka
Mobil berhasil ditambahkan
Mobil berhasil masuk gudang!

```

Jika pengguna memilih angka 2, pengguna akan diminta untuk mengisi data-data, setelahnya sistem akan memproses data tersebut untuk membuat objek baru lalu menyimpannya dalam garasi array. Untuk melihat mobil yang tersedia setelah ditambahkan dapat memilih menu 1. Jika terdapat pesan gagal atau tidak berhasil

menambahkan mobil maka artinya gudang sudah penuh atau array yang di patok 10 sudah mencapai maksimalnya.

```
=== Simulasi Dealer ===
Stok: 2/10 mobil
1. Lihat stok mobil
2. Mobil baru datang (tambah stok)
3. Pelanggan membeli mobil (Hapus stok)
4. Keluar:
Pilih menu:
3

Hapus mobil karena dibeli
Mobil apa yang dibeli: Brio
Berhasil menghapus mobil

=== Simulasi Dealer ===
Stok: 1/10 mobil
1. Lihat stok mobil
2. Mobil baru datang (tambah stok)
3. Pelanggan membeli mobil (Hapus stok)
4. Keluar:
Pilih menu:
1
Stok Mobil Tersedia
=== Daftar Mobil ===
Mobil ke-1:
Nama: Civic
Tahun: 2019
CC: 1500
Harga: 250000000
Merek: Honda

=====
```

Jika pengguna memilih angka 3, sistem akan meminta nama mobil yang ingin dihapus nama mobil yang sudah dibeli pelanggan.

```
=== Simulasi Dealer ===
Stok: 1/10 mobil
1. Lihat stok mobil
2. Mobil baru datang (tambah stok)
3. Pelanggan membeli mobil (Hapus stok)
4. Keluar:
Pilih menu:
4
Terima kasih
```

Pada menu ke 4, program akan menghentikan perulangan menu dan mencetak pesan terima kasih sebagai penutup.